

Engineering Document E01 — HQ Monorepo Architecture

Product Name: HQ

Engineering Package: Version 1.0

Purpose

This document defines the official monorepo architecture for HQ.

The repository is designed to support:

- Web application
- Mobile application
- Backend API
- AI Orchestration Engine
- Shared packages
- Documentation
- Infrastructure
- Future microservices

The architecture prioritizes scalability, maintainability, and developer productivity.

Engineering Principles

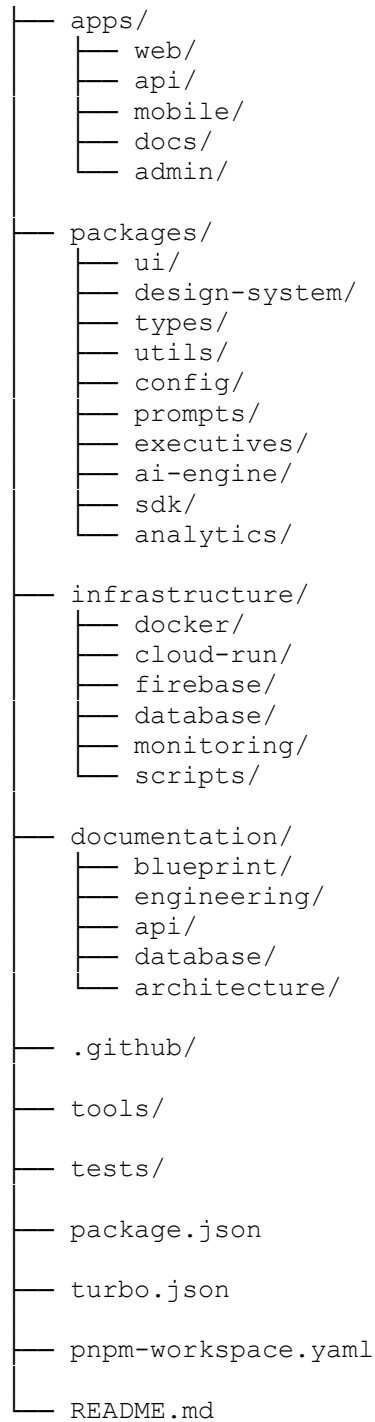
The repository should be:

- Modular
- Scalable
- Type-safe
- Testable
- Reusable
- Consistent
- Easy to navigate

Every package has a single responsibility.

Repository Structure

hq/



Applications

apps/web

Purpose:

The primary HQ web application built with Next.js.

Responsibilities:

- Landing page
 - Authentication
 - Dashboard
 - Executive Board
 - Mission Control
 - Billing
 - Analytics
 - User settings
-

apps/api

Purpose:

NestJS backend.

Responsibilities:

- REST APIs
 - Authentication
 - Database access
 - AI orchestration
 - Billing
 - Notifications
 - Executive routing
-

apps/mobile

Purpose:

Future React Native application.

Version 1.0 does not ship the mobile app, but the folder exists to support future expansion without restructuring the repository.

[apps/docs](#)

Purpose:

Documentation website.

Contains:

- API documentation
 - Developer guides
 - SDK documentation
 - Architecture references
-

[apps/admin](#)

Purpose:

Internal administration portal.

Functions:

- User management
 - Subscription management
 - AI monitoring
 - Executive analytics
 - Platform administration
-

Shared Packages

[packages/ui](#)

Reusable React components.

Examples:

- Button
 - Card
 - Sidebar
 - Modal
 - Table
 - Form controls
 - Charts
-

[packages/design-system](#)

Contains:

- Colors
 - Typography
 - Icons
 - Spacing
 - Motion tokens
 - Themes
-

[packages/types](#)

Shared TypeScript types.

Examples:

- User
 - Mission
 - Executive
 - Subscription
 - Organization
 - API responses
-

[packages/utils](#)

Reusable utilities.

Examples:

- Date formatting
 - Validation
 - Formatting
 - Helpers
 - Constants
-

[packages/config](#)

Centralized configuration.

Contains:

- Environment configuration
 - Feature flags
 - Application constants
-

[packages/prompts](#)

The production Prompt Library.

Contains:

- CEO prompts
 - Executive prompts
 - Review prompts
 - Workflow prompts
 - Prompt versions
-

[packages/executives](#)

Business logic for every AI executive.

Each executive has:

- Identity
- Responsibilities

- Input schema
 - Output schema
 - Validation rules
-

[packages/ai-engine](#)

Core orchestration engine.

Responsibilities:

- Mission routing
 - Planning
 - Memory
 - AI provider integration
 - Executive collaboration
 - Quality review
-

[packages/sdk](#)

Reusable SDK for future integrations.

Supports:

- API clients
 - Authentication helpers
 - Webhooks
-

[packages/analytics](#)

Shared analytics logic.

Provides:

- Event tracking
 - Usage metrics
 - Dashboard calculations
-

Infrastructure

The infrastructure directory contains deployment resources.

Includes:

- Docker
- Cloud Run
- Database migrations
- Monitoring
- Automation scripts

Infrastructure is version-controlled alongside application code.

Documentation

The documentation directory stores:

- Product Blueprint
- Engineering Documents
- API Specifications
- Database Design
- ADRs (Architecture Decision Records)

Documentation evolves with the codebase.

Testing

Centralized testing includes:

- Unit tests
- Integration tests
- End-to-end tests
- Performance tests
- AI evaluation tests

Testing standards apply across all applications and packages.

Dependency Rules

Applications may depend on shared packages.

Shared packages must not depend on applications.

Dependencies should always point inward toward reusable code.

Naming Conventions

Directories:

- kebab-case

Files:

- kebab-case

React Components:

- PascalCase

Functions:

- camelCase

Classes:

- PascalCase

Constants:

- UPPER_SNAKE_CASE

Environment Variables:

- UPPER_SNAKE_CASE

Consistency is mandatory across the repository.

Package Ownership

Each package has:

- Purpose
- Maintainer
- Version
- Documentation
- Tests

Ownership promotes accountability and maintainability.

Build System

Recommended tools:

- pnpm Workspaces
- Turborepo
- TypeScript
- ESLint
- Prettier
- Husky
- lint-staged

These tools provide fast builds and consistent code quality.

Versioning Strategy

Follow Semantic Versioning:

- Major
- Minor
- Patch

Breaking changes require a major version increment.

Repository Standards

Every package should contain:

- README
- Source code
- Tests
- Configuration
- Documentation

Packages without documentation should not be merged into the main branch.

Future Expansion

The architecture supports:

- Desktop application
- Browser extension
- AI Marketplace
- Plugin ecosystem
- Enterprise integrations
- Additional AI providers

These additions should integrate without major restructuring.

Success Criteria

The Monorepo Architecture succeeds when:

- Developers can quickly locate code.
- Shared logic is reusable.
- Applications remain independent.
- Scaling does not require repository reorganization.
- New features can be added with minimal friction.

Conclusion

The HQ Monorepo Architecture provides a scalable engineering foundation for the platform.

By separating applications, shared packages, infrastructure, and documentation into clearly defined modules, HQ can evolve from a hackathon MVP into a production-grade AI platform without sacrificing maintainability or developer experience.